Model Context Protocol (MCP): Avaliação da Qualidade de Descrições de Tools com Base em Contexto de Código Fonte

Pedro H. P. Rodrigues[1], **Vinicius R. A. de A. Moreira**[1]

[1]Instituto de Ciências Exatas e Informática – Pontifícia Universidade Católica de Minas Gerais
Caixa Postal 500 – 30535-901 – Belo Horizonte – MG – Brazil

{vinicius,pedro}viniciusarantes133@gmail.com, pedrohenriquepr08@gmail.com

Resumo. *O Model Context Protocol (MCP) permite que agentes de Inteligência Artificial (IA) interajam com sistemas externos por meio de tools, cujas descrições em linguagem natural orientam sua utilização. Descrições de baixa qualidade, caracterizadas pela presença de smells, podem comprometer a eficácia desses agentes. Este trabalho investiga se a avaliação da qualidade dessas descrições muda quando o avaliador tem acesso ao código da ferramenta. Para isso, 12.171 tools de 206 servidores MCP do GitHub são extraídas, junto com o código alcançado por sua árvore de chamadas, e avaliadas por três LLMs, segundo uma rubrica de seis dimensões, com e sem esse código. As diferenças entre os cenários são estatisticamente significativas nas seis dimensões, mas sua direção depende do avaliador, e, na maioria das ferramentas em que todos os avaliadores mudam a nota, a mudança decorre de informações que o código revela e a descrição omite, como parâmetros não documentados e modos de falha não divulgados.*

Abstract. *The Model Context Protocol (MCP) enables Artificial Intelligence (AI) agents to interact with external systems through tools, whose natural language descriptions guide their usage. Poor-quality descriptions, characterized by the presence of smells, can compromise the effectiveness of these agents. This work investigates whether the quality assessment of these descriptions changes when the evaluator has access to the tool's source code. To this end, 12,171 tools from 206 MCP servers on GitHub are extracted, together with the code reached by their call graph, and evaluated by three LLMs against a six-dimension rubric, with and without that code. The differences between scenarios are statistically significant in all six dimensions, but their direction depends on the evaluator, and, for most tools whose score changes across all evaluators, the change stems from information the code reveals and the description omits, such as undocumented parameters and undisclosed failure modes.*

Bacharelado em Engenharia de *Software* - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): João Pedro Batisteli - joaobatisteli@pucminas.br
Orientador acadêmico (TCC I): Leonardo Vilela - leonardocardoso@pucminas.br
Orientador do TCC II: Glender Brás - glendermedeiros@pucminas.br

Belo Horizonte, 28 de junho de 2026.

1. Introdução

O avanço dos modelos de linguagem (*Large Language Models*, LLMs) transformou o paradigma de interação entre humanos e máquinas [Hasan et al. 2026]. No entanto, a ausência inicial de padrões de comunicação entre LLMs e ferramentas externas gerou uma fragmentação técnica que motivou o surgimento do *Model Context Protocol* (MCP), um protocolo aberto criado para padronizar a integração de modelos de IA com programas e fontes de dados externas [Shi et al. 2026]. A adoção do protocolo tem crescido de forma acelerada, e um levantamento publicado no início de 2026 apontou que seus Kits de Desenvolvimento de Software (*Software Development Kit*, SDK) já registravam aproximadamente 97 milhões de *downloads* mensais [LLC 2026]. O principal diferencial do MCP reside no uso de ferramentas (*tools*) descritas em linguagem natural, que o agente pode selecionar e invocar em tempo de execução conforme a tarefa a ser resolvida. Esse funcionamento impõe, contudo, uma dependência crítica da qualidade documental, uma vez que o modelo não tem acesso ao código que implementa as ferramentas. As decisões de usá-las, como invocá-las e quais parâmetros fornecer são tomadas com base em suas descrições [Shi et al. 2026, Luo et al. 2025]. Dessa forma, assim como *code smells* comprometem a manutenibilidade de *softwares* tradicionais, descrições ambíguas ou incompletas comprometem o comportamento do agente, podendo induzir erros de planejamento, uso incorreto de parâmetros e alucinações [Mehedi Hasan et al. 2026].

Estudos recentes investigam a prevalência de anomalias (ou *smells*) em descrições de ferramentas MCP por meio de análise documental [Mehedi Hasan et al. 2026][Guo et al. 2025, Shi et al. 2026]. Essa abordagem avalia se a documentação comunica adequadamente o propósito, os parâmetros e as restrições de uso de uma ferramenta, sem considerar a implementação técnica [Hasan et al. 2026]. Entretanto, falhas relacionadas à configuração e à documentação representam 38,66% dos problemas observados em servidores MCP [Taraghi et al. 2026]. No MCP, a descrição é um artefato redigido manualmente para um código já existente, e é o código que define o comportamento efetivo da ferramenta; a prevalência de inconsistências entre os dois em servidores reais [Shi et al. 2026] mostra que eles podem divergir. Diante disso, existe uma lacuna na **análise da qualidade das descrições das ferramentas MCP, que, sem o contexto do código, pode não ser suficiente para a avaliação de sua qualidade**.

Essa lacuna é relevante porque inconsistências semânticas podem reduzir a eficácia dos agentes e ampliar a superfície de exposição a vulnerabilidades, incluindo ataques de injeção de comandos [Taraghi et al. 2026], e relatórios de segurança alertam que ambiguidades documentais podem favorecer a execução de ações não autorizadas ou a exposição indevida de dados sensíveis [National Security Agency 2026].

Tendo em vista essa lacuna, este trabalho tem como objetivo geral **investigar se a avaliação da qualidade de descrições de ferramentas MCP é influenciada pela disponibilização do código da ferramenta ao avaliador.** Para alcançar esse objetivo, definem-se os seguintes objetivos específicos: (i) Examinar se o fornecimento do código da *tool* resulta em maior precisão na avaliação da qualidade de sua descrição; (ii) Identificar os atributos de avaliação mais afetados pela adição do código da *tool*; e (iii) Realizar uma análise estatística sobre os resultados obtidos nas avaliações.

Esses objetivos são orientados pelas seguintes questões de pesquisa: **RQ1**: o fornecimento do código da *tool* ao avaliador aumenta a precisão da avaliação da qualidade

de sua descrição, em relação à avaliação baseada apenas na descrição? **RQ2**: quais atributos da rubrica de qualidade documental são mais afetados pela adição do código da *tool*? **RQ3**: as diferenças observadas entre os escores dos cenários com e sem código são estatisticamente significativas?

Para respondê-las, as ferramentas de 206 servidores MCP de código aberto são avaliadas por três modelos de linguagem atuando como avaliadores (*LLM-as-a-Judge*), que pontuam, em escala Likert de cinco pontos, as seis dimensões da rubrica de qualidade documental de Hasan et al. (2026), com e sem acesso ao código da ferramenta.

As próximas seções deste artigo estão organizadas da seguinte forma. A Seção 2 apresenta a Fundamentação Teórica. A Seção 3 discute os trabalhos relacionados. A Seção 4 descreve os materiais e métodos, a Seção 5 apresenta os resultados, a Seção 6 discute as ameaças à validade e a Seção 7 apresenta as conclusões e os trabalhos futuros.

2. Fundamentação Teórica

2.1. Qualidade de Software

A qualidade, no âmbito da Engenharia de Software, pode ser compreendida como o grau em que um produto de software atende aos requisitos especificados e às necessidades e expectativas de seus usuários e demais partes interessadas [ISO/IEC/IEEE 2017]. Segundo Pressman e Maxim (2016), a qualidade de software envolve tanto a conformidade com requisitos funcionais e de desempenho explicitamente declarados quanto a aderência a padrões de desenvolvimento e a características implícitas esperadas de todo software desenvolvido profissionalmente. Normas internacionais como a ISO/IEC 25010 [ISO/IEC 2011] estabelecem modelos de qualidade que decompõem esse conceito em características mensuráveis, tais como adequação funcional, confiabilidade, usabilidade e manutenibilidade. Essa avaliação não se restringe ao código, estendendo-se também aos artefatos que compõem o software, como documentação, especificações e descrições, cuja clareza e precisão impactam diretamente a compreensão e o uso correto do sistema. Quando quem consome a documentação é um agente de IA, que decide a partir dela quando e como agir, uma descrição de baixa qualidade deixa de ser uma falha documental isolada e passa a ser um defeito com efeito comportamental observável; é nessa dimensão que este trabalho se concentra.

2.2. Model Context Protocol

O *Model Context Protocol* (MCP) é um padrão aberto que define como aplicações de IA se conectam a fontes de dados e ferramentas externas, de modo análogo ao papel que o USB-C desempenha ao uniformizar a conexão de periféricos de *hardware* [Guo et al. 2025]. Tecnicamente, o protocolo adota uma arquitetura cliente-servidor baseada em mensagens JSON-RPC, na qual um *host* orquestra um ou mais servidores [Shi et al. 2026]. Cada servidor disponibiliza suas capacidades por meio de três primitivas: as ferramentas (*tools*), que executam ações; os recursos (*resources*), que fornecem dados contextuais; e os *prompts*, que oferecem modelos de instrução [Guo et al. 2025]. Esse modelo padronizado dispensa código de integração específico para cada plataforma e favorece a interoperabilidade e o reúso de ferramentas no ecossistema [Hasan et al. 2026].

2.3. Descrições de Ferramentas e *Smells*

No MCP, as ferramentas dependem de descrições em linguagem natural como principal guia para os agentes. Diferentemente de sistemas tradicionais, que utilizam assinaturas de código estritas, a orquestração por IA exige que o propósito, os parâmetros e as restrições de cada funcionalidade sejam explicados textualmente [Mehedi Hasan et al. 2026]. Uma descrição ambígua ou incompleta pode induzir o modelo a erros de planejamento, resultando em chamadas inválidas ou no uso de parâmetros inadequados [Shi et al. 2026]; a descrição funciona, assim, como um artefato de especificação.

O conceito de *smell* refere-se a características estruturais que sugerem problemas de qualidade em um artefato, ainda que não configurem defeitos funcionais [Mehedi Hasan et al. 2026]. Embora originalmente associado ao código, o conceito foi estendido a artefatos em linguagem natural, como os *requirements smells*, que identificam ambiguidades em especificações de requisitos [Femmer et al. 2017], e os *documentation smells*, que apontam problemas de qualidade em documentação de APIs [Khan et al. 2021]. No âmbito do MCP, o termo aplica-se a ambiguidades, inconsistências e omissões nas descrições de ferramentas, como a ausência de especificação dos parâmetros, do comportamento esperado e das limitações da ferramenta [Mehedi Hasan et al. 2026]. Por exemplo, uma ferramenta cuja descrição se limita a "processa os dados" configura um *smell* de vagueza: o texto não especifica quais dados são aceitos, qual transformação é aplicada nem qual saída é produzida.

3. Trabalhos Relacionados

Esta seção apresenta estudos sobre a qualidade das descrições MCP, o desempenho de agentes no ecossistema e o uso de LLMs como avaliadores, que fundamentam a lacuna desta pesquisa.

Hasan et al. (2026) investigaram a qualidade das documentações de 856 ferramentas no ecossistema MCP por meio de uma análise empírica fundamentada em LLMs. O estudo aplicou uma rubrica de seis dimensões focada na clareza e completude do conteúdo textual. Os resultados revelaram que 97,1% das descrições possuem ao menos um *smell*, o que reduz a precisão dos agentes na seleção de parâmetros. O presente estudo parte dessa investigação e avança em uma direção não explorada pelos autores: a incorporação do código ao processo de avaliação, verificando em que medida o acesso à lógica interna do servidor altera os escores obtidos nas dimensões da rubrica proposta pelos autores.

Shi et al. (2026) investigam a prevalência e os impactos de inconsistências semânticas entre descrições de ferramentas MCP e seus respectivos códigos, propondo uma taxonomia de inconsistências e um detector baseado em LLM aplicado em larga escala a servidores reais. Os resultados revelaram que 9,93% das ferramentas analisadas apresentam inconsistências e que 35% dos servidores possuem ao menos um caso, com riscos de seleção incorreta de ferramentas, efeitos colaterais não declarados e falhas de segurança. Esse estudo sustenta a premissa desta pesquisa ao demonstrar que informações relevantes sobre o comportamento de uma ferramenta frequentemente não estão presentes em sua descrição.

Guo et al. (2025) mapeiam a adoção do MCP com um rastreador que analisa mais de 8.400 projetos e constatam que mais da metade dos projetos listados são inválidos,

abandonados ou de baixo valor agregado. O estudo caracteriza os repositórios, mas não examina a consistência entre descrições e implementação; o dado de baixa maturidade também fundamenta o critério de popularidade adotado na seleção da amostra (Seção 4).

Luo et al. (2025) propuseram o MCP-Universe, um *benchmark* que avalia modelos de linguagem em servidores MCP reais por meio de avaliadores baseados em execução. Ao avaliar mais de 20 modelos em 231 tarefas distribuídas por 11 servidores, os resultados revelaram que mesmo modelos de fronteira atingem taxas de sucesso abaixo de 45%, com falhas frequentemente associadas ao uso incorreto de interfaces de ferramentas pouco familiares. O presente trabalho não mede o desempenho de agentes; ele examina um fator anterior a essas falhas, a qualidade da informação que a descrição oferece ao agente sobre a interface da ferramenta, e se o código revela omissões nessa informação.

Zheng et al. (2023) investigam a viabilidade da abordagem *LLM-as-a-Judge*, comparando julgamentos do GPT-4 com avaliações humanas em *benchmarks* de conversação, e mostram concordância superior a 80%, comparável à observada entre os próprios especialistas. Esse resultado fundamenta o uso de avaliadores baseados em LLM neste estudo.

Em síntese, é demonstrado que a qualidade das descrições MCP é um fator crítico e que texto e implementação podem divergir, mas as abordagens de avaliação de qualidade analisam as documentações de forma isolada. Este trabalho investiga como a incorporação do código influencia essa avaliação.

4. Materiais e Métodos

A pesquisa caracteriza-se como um estudo empírico, quantitativo e comparativo, cujo fluxo está resumido na Figura 1. O processo inicia-se com a coleta de repositórios MCP públicos, seguida pela extração das ferramentas e do código que as implementa. Em seguida, a qualidade das descrições é avaliada em dois cenários: no primeiro, apenas com a descrição em linguagem natural; no segundo, com a mesma descrição acompanhada pelo código correspondente. Por fim, os resultados são submetidos a uma análise comparativa, estatística e qualitativa.

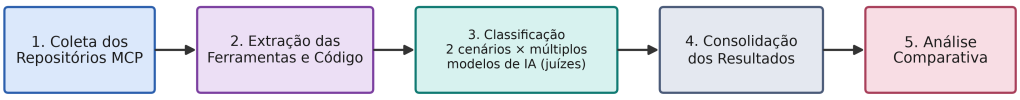


Figura 1. Fluxo metodológico adotado na pesquisa

4.1. Coleta dos Repositórios MCP

Foram minerados repositórios públicos de servidores MCP no GitHub. Como a API REST de busca de código limita cada consulta a 1.000 resultados [Dong et al. 2023], a busca foi segmentada em uma subconsulta por combinação entre um pacote do SDK MCP (por exemplo, `@modelcontextprotocol/sdk`) e um arquivo de manifesto em que ele pode ser declarado (por exemplo, `package.json`). Metadados ausentes na busca de código (estrelas, condição de *fork* e linguagem principal) foram obtidos pela

API GraphQL, e os resultados foram deduplicados por identificador. Dos 12.058 candidatos brutos, 723 (6,0%) atenderam aos critérios de elegibilidade: não ser *fork*, estar em uma linguagem-alvo e possuir ao menos 100 estrelas, limiar usado na mineração de repositórios para excluir projetos experimentais ou de baixo valor agregado, os *toy repositories* [Borges et al. 2016]. Os elegíveis foram ordenados por estrelas até completar 206 servidores, o dobro da amostra de Hasan et al. (2026); um candidato só permanecia na amostra se a Etapa 2 localizasse nele ao menos uma ferramenta, sendo substituído pelo próximo da lista caso contrário (laço de *backfill*). A Figura 2 detalha o funcionamento dessa etapa.

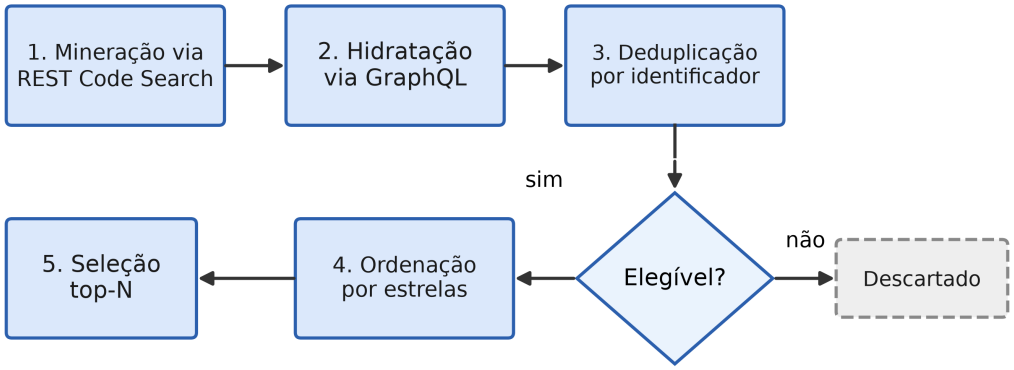


Figura 2. Funcionamento detalhado da Etapa 1 (Coleta dos Repositórios)

4.2. Extração das Ferramentas e do Código Relacionado

Cada repositório selecionado foi clonado localmente e analisado por um algoritmo de mineração baseado em análise estática (*tree-sitter*), que localiza as ferramentas declaradas segundo o padrão do SDK MCP de cada linguagem e extrai sua descrição em linguagem natural e sua localização no código, conforme detalhado na Figura 3.

Como os servidores MCP possuem estruturas variadas, extrair apenas o método raiz de uma ferramenta pode não ser suficiente para capturar sua lógica. Para isso, foi construída uma árvore de chamadas (*call graph*) a partir do método inicial, expandida recursivamente até três níveis de profundidade: o próprio método constitui o primeiro nível, os métodos chamados diretamente por ele, quando resolvidos, formam o segundo, e as chamadas efetuadas a partir destes compõem o terceiro. O limite de três níveis contém o custo da análise e do texto enviado ao avaliador; a heurística de resolução está detalhada no Apêndice A. Um exemplo é apresentado na Figura 4.

A Etapa 2 extraiu 12.171 ferramentas dos 206 repositórios (Tabela 1). Python e TypeScript concentram 60% dos repositórios e 85,6% das ferramentas, e a distribuição por repositório é fortemente assimétrica: os três repositórios com mais ferramentas respondem por 46,6% do conjunto (20,4%, 18,4% e 7,8%), o que deve ser considerado ao interpretar médias agregadas.

4.3. Avaliação das Ferramentas e Consolidação dos Resultados

Cada ferramenta foi avaliada em dois cenários: (i) sem código, recebendo nome, servidor de origem e descrição; e (ii) com código, recebendo adicionalmente os trechos de código-

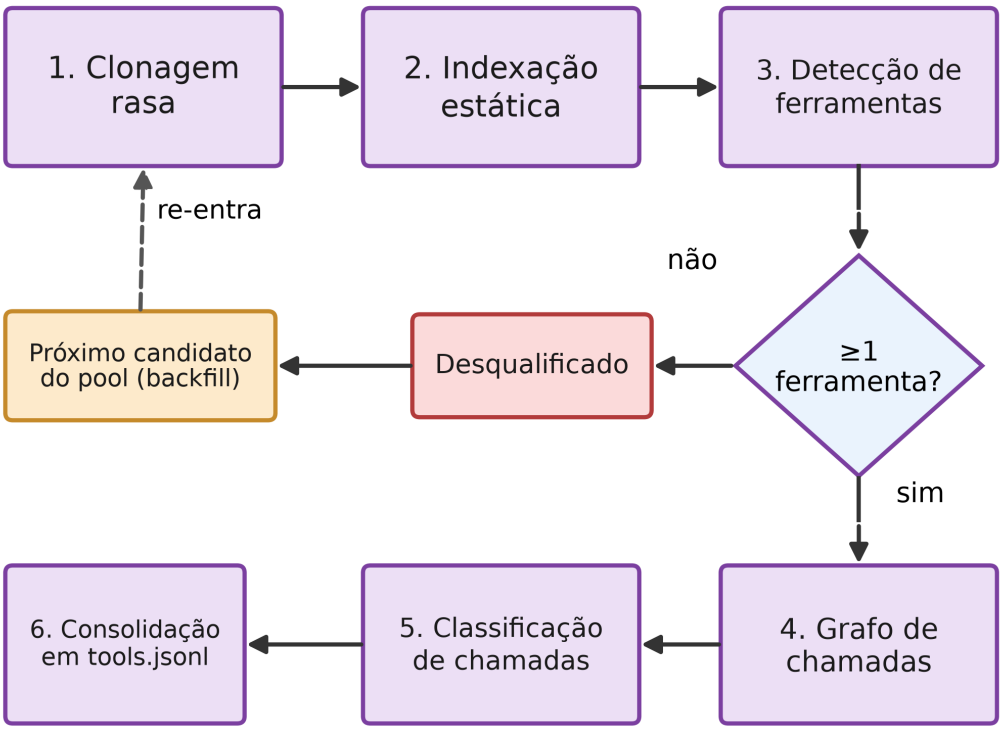


Figura 3. Funcionamento detalhado da Etapa 2 (Extração das Ferramentas)

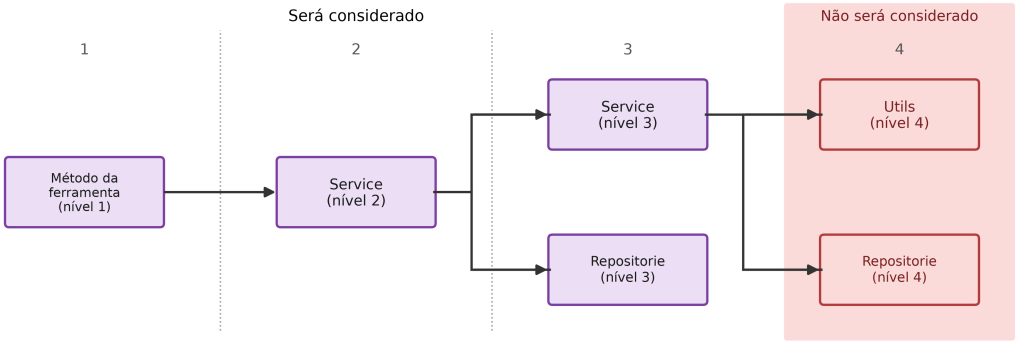


Figura 4. Exemplo de extração da árvore de chamadas de uma ferramenta

Tabela 1. Repositórios selecionados e total de ferramentas extraídas, por linguagem

Linguagem	Repositórios	Tools (total)	Média/repositório
Python	79	7.251	91,78
TypeScript	45	3.162	70,27
Java	13	793	61,00
C#	21	423	20,14
Rust	26	276	10,62
Go	13	149	11,46
JavaScript	6	108	18,00
Ruby	2	5	2,50
Dart	1	4	4,00
Total	**206**	**12.171**	**59,08**

fonte reunidos pela árvore de chamadas (Seção 4.2). A descrição é a mesma nos dois cenários; a única variável é a presença do código. A Figura 5 detalha o funcionamento dessa etapa.

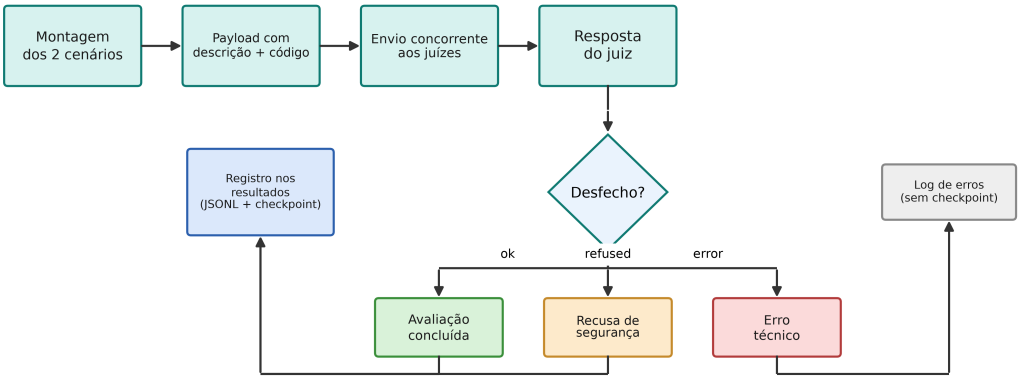


Figura 5. Funcionamento detalhado da Etapa 3 (Classificação das Ferramentas)

Em ambos, modelos de linguagem atuam como juízes (*LLM-as-a-Judge*) e pontuam, em escala Likert de cinco pontos, os seis componentes da rubrica de Hasan et al. (2026): *Purpose* (clareza da finalidade da ferramenta), *Guidelines* (orientações sobre quando e como utilizá-la), *Limitations* (restrições, premissas e casos de falha), *Parameter Explanation* (documentação dos parâmetros de entrada), *Length and Completeness* (detalhamento e cobertura) e *Examples* (presença de exemplos de uso).

Foram utilizados três juízes de proveniências e custos distintos: DeepSeek-V4.1-Flash e Gemini 3.5 Flash-Lite, acessados via API, e Qwen3-14B, executado localmente via Ollama. A combinação reduz o risco de que um padrão observado seja específico de um único provedor, e a comparação entre eles é tratada como um achado, não como um pressuposto. As notas foram consolidadas em uma base pareada por ferramenta, juiz e cenário. Dela deriva-se a **amostra de divergência confirmada**: as ferramentas em que, para ao menos um componente da rubrica, todos os juízes atribuem notas diferentes entre

os cenários sem e com código. Para cada uma, reúnem-se a justificativa (*reasoning*) de cada juiz nos dois cenários e o código correspondente.

4.4. Análise Comparativa dos Experimentos

Para cada juiz e componente, calculam-se a proporção de ferramentas cuja nota diminuiu, se manteve ou aumentou com a inclusão do código e a distribuição das notas nos dois cenários, identificando os atributos mais afetados (objetivo específico ii). Para verificar a significância das diferenças (objetivo específico iii), as notas dos três juízes são agregadas pela média, por ferramenta e componente, e aplica-se a cada componente o Teste de Postos Sinalizados de Wilcoxon bilateral [Wilcoxon 1945], adequado a amostras pareadas sem suposição de normalidade, com nível de significância de 5% ($\alpha = 0{,}05$) e sem correção para comparações múltiplas; pares com diferença nula são descartados, conforme o tratamento padrão de empates do teste, e o número de pares com diferença (n) é reportado. Complementarmente, aplica-se uma análise de transição de quartil: os quartis da distribuição da nota média no cenário sem código definem quatro faixas por componente, e cada ferramenta é reclassificada segundo a faixa em que sua nota média com código se enquadra, reportando-se a proporção que mantém, sobe ou desce de faixa.

Para caracterizar que tipo de informação o código revela, conduz-se uma análise qualitativa sobre a amostra de divergência confirmada (Figura 6; *prompts* no Apêndice B). Nas fases de familiarização e codificação inicial de uma análise temática [Braun and Clarke 2006], um modelo de linguagem (Gemini) confronta, para cada ferramenta, as justificativas dos juízes com o código e sintetiza uma justificativa única sobre o que o código revelou e a descrição omitia, sendo instruído a registrar a ausência de evidência em vez de inventar uma explicação. Nas fases de busca, revisão e definição de temas, um segundo modelo (DeepSeek) recebe o conjunto completo de justificativas em uma única chamada e induz, de baixo para cima, categorias de mecanismo, cada uma sustentada por mais de uma ferramenta. Em seguida, uma análise de conteúdo dirigida [Hsieh and Shannon 2005] classifica cada justificativa em zero ou mais categorias, uma ferramenta por vez, para evitar a degradação de desempenho em listas longas [Liu et al. 2024].

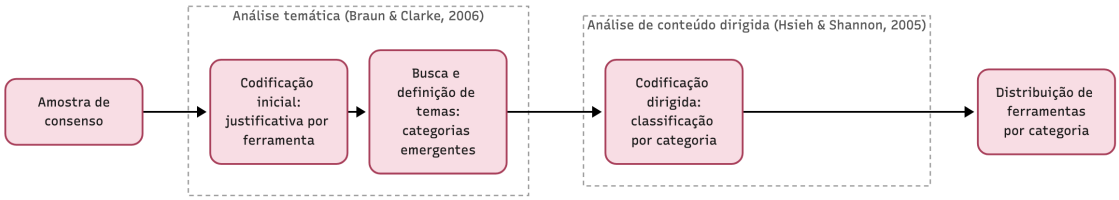


Figura 6. Fluxo detalhado da análise qualitativa sobre a amostra de divergência confirmada

Por fim, a classificação é verificada manualmente em 100 ferramentas, tamanho dado pela Fórmula de Cochran para populações finitas [Chaokromthong et al. 2021] com 95% de confiança, proporção estimada de 0,5 e margem de erro de 10% ($n = 96$, arredondado para 100), como em Hasan et al. (2026). A amostra é estratificada com vagas distribuídas igualmente entre as categorias (mais `none`), priorizando repositórios e componentes ainda não representados, e a categoria atribuída é comparada com uma leitura independente da justificativa sintetizada e do *reasoning* original dos juízes.

5. Resultados

Esta seção apresenta os resultados da avaliação por rubrica descrita na Seção 4, com base nas avaliações dos três juízes, DeepSeek-V4.1-Flash, Gemini 3.5 Flash-Lite e Qwen3-14B, que avaliaram as mesmas 12.170 ferramentas nos dois cenários experimentais (sem código e com código). Das 12.171 ferramentas extraídas, uma não obteve avaliação pareada válida do Qwen3-14B e foi excluída de todas as análises, para que os três juízes sejam comparados sobre exatamente o mesmo conjunto.

5.1. RQ1: O fornecimento do código da *tool* ao avaliador aumenta a precisão da avaliação da qualidade de sua descrição, em relação à avaliação baseada apenas na descrição?

O aumento ou a diminuição da nota não indica, por si só, uma melhora ou piora na precisão da avaliação: a divergência pode refletir tanto um ganho real de precisão quanto viés ou ruído do avaliador. Responder à RQ1 exige, portanto, identificar o **mecanismo** pelo qual o código altera a nota, não apenas a direção da mudança.

A análise qualitativa (Seção 4.4) foi conduzida sobre as 2.273 ferramentas da amostra de divergência confirmada (18,7% das 12.170 avaliadas). A análise temática indutiva produziu 16 categorias de mecanismo, cada uma descrevendo um tipo de informação que o código revela e que a descrição não comunica; suas definições estão no Apêndice C. A classificação dirigida, que permite múltiplas categorias por ferramenta, deixou apenas 4,1% das ferramentas (94 de 2.273) sem nenhuma categoria (`none`), indicando que o conjunto cobre a grande maioria dos mecanismos presentes. A Tabela 2 apresenta a frequência de cada categoria.

Tabela 2. Frequência de cada categoria de mecanismo na amostra de divergência confirmada (n = 2.273; multi-*label*, a soma excede 100%)

Categoria	Ocorrências	%
Superfície de Entrada Não Documentada	1.304	57,4%
Modos de Falha e Erros Não Divulgados	592	26,1%
Estrutura Concreta da Saída	443	19,5%
Regras de Validação e Contratos de Formato Não Divulgados	435	19,1%
Limites Operacionais, Padrões e Paginação Não Divulgados	386	17,0%
Contexto de Fluxo e Ativação	291	12,8%
Escopo de Processamento e Mecânica Interna Subestimados	278	12,2%
Confirmação de Encapsulamento Trivial	217	9,6%
Valores Fixos e Padrões Divergentes	183	8,1%
Dependências e Pré-requisitos de Ambiente Ocultos	165	7,3%
Semântica de Correspondência e Normalização de Entrada	109	4,8%
Controles de Acesso, Papel e Autorização	103	4,5%
Parâmetros Fantasmas ou Divergentes	79	3,5%
Funcionalidade Anunciada Não Implementada	73	3,2%
Sistema-Alvo Divergente	58	2,6%
Efeitos Colaterais Não Divulgados	56	2,5%
(nenhuma categoria)	94	4,1%

Superfície de Entrada Não Documentada domina a amostra (57,4%): o achado mais comum é que o código aceita parâmetros, filtros ou opções que a descrição não menciona. Modos de Falha e Erros Não Divulgados (26,1%) e Estrutura Concreta da

Saída (19,5%) seguem como segundo e terceiro mecanismos mais recorrentes, e mesmo a categoria menos frequente, Efeitos Colaterais Não Divulgados, aparece em 56 ferramentas.

Na verificação manual, os dois autores avaliaram individualmente as categorizações atribuídas pelo modelo às 100 ferramentas da amostra, marcando cada uma como "concordo" ou "discordo". Ao final, o avaliador A concordou com 86% das categorizações do modelo e o avaliador B, com 80%. Entre os dois avaliadores, houve divergência em 30% das respostas. Esses resultados indicam que a categorização realizada pelo modelo de IA é, em geral, confiável.

Em resposta à RQ1, em 89,2% das ferramentas da amostra (2.028 de 2.273) a mudança de nota é rastreável a um achado concreto no código; nos 10,8% restantes (sem categoria ou apenas Confirmação de Encapsulamento Trivial), o código não revela informação nova e a mudança reflete apenas uma releitura mais rigorosa do mesmo texto. A mudança de nota tem, portanto, causa identificável na maioria dos casos, o que é consistente com um ganho de precisão, mas não o comprova na ausência de um padrão-ouro humano (Seção 6).

5.2. RQ2: Quais atributos da rubrica de qualidade documental são mais afetados pela adição do código da *tool*?

A Figura 7 apresenta, para cada juiz, a proporção de casos em que a avaliação com código resultou em diminuição, manutenção ou aumento da nota de cada atributo, e a Figura 8 compara a distribuição das notas nos dois cenários.

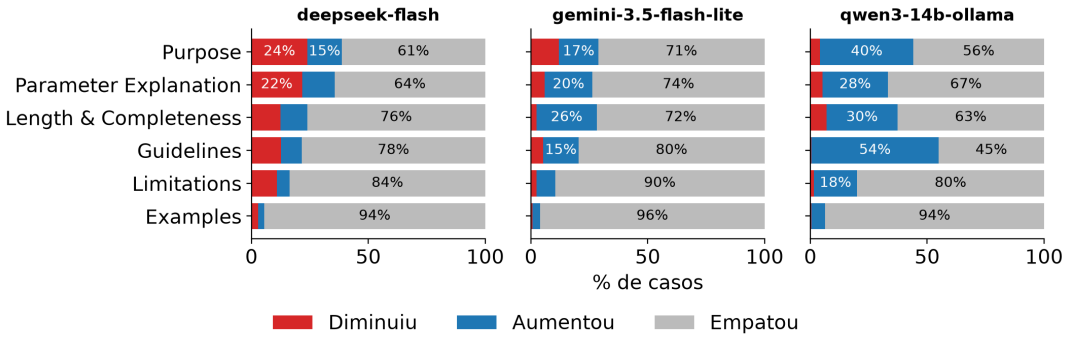


Figura 7. Distribuição proporcional da alteração nas pontuações por atributo e por juiz.

Três padrões se destacam. Primeiro, a direção da mudança depende do juiz: o DeepSeek tende a reduzir a nota com o código, com queda da média em todos os atributos e da mediana em *Purpose*, *Parameter Explanation* e *Limitations*, enquanto Gemini e Qwen3 tendem a elevá-la, com média maior em todos os atributos e mediana maior em todos, exceto *Guidelines* no Gemini; o Qwen3 apresenta as mudanças mais extremas, o que indica menor concordância com os outros dois juízes. Segundo, *Purpose* é, em média, o atributo com maior variação, enquanto *Limitations* e *Examples* variam menos nos três juízes. Terceiro, *Guidelines* destoa no Qwen3: é o único atributo em que os aumentos de nota superam os empates, embora seja o terceiro com menos variação no DeepSeek e no Gemini.

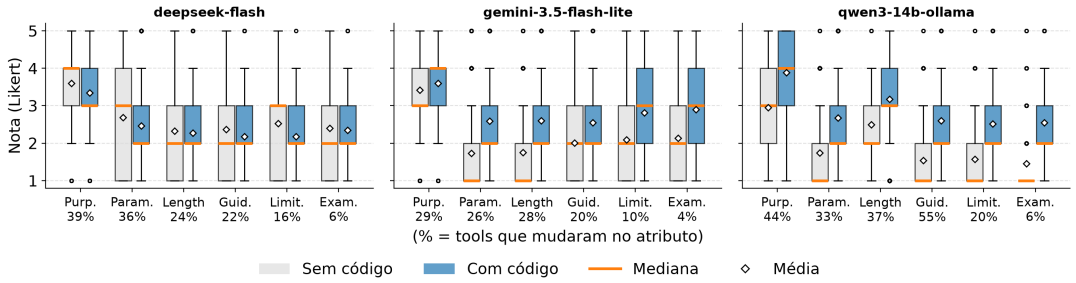


Figura 8. Distribuição das notas nos cenários com e sem código, por juiz, considerando apenas os casos com alteração de pontuação (a maioria das avaliações empata, o que ocultaria a diferença).

5.3. RQ3: As diferenças observadas entre os escores dos cenários com e sem código são estatisticamente significativas?

A Tabela 3 reúne, por atributo, o resultado do Teste de Wilcoxon e da transição de quartil, ambos calculados sobre a média das notas dos três juízes (Seção 4.4).

Tabela 3. Teste de Postos Sinalizados de Wilcoxon e transição de quartil das notas com e sem código, por atributo (média dos três juízes; pareado por *tool*). n: pares com diferença não nula; faixas de quartil definidas pela distribuição sem código.

	Wilcoxon			Quartil		
Atributo	n	W	p-valor	Manteve	Subiu	Desceu
Purpose	7675	8576765,0	$1,49 \times 10^{-224}$	68,2%	21,6%	10,2%
Parameter Explanation	7102	7716300,5	$5,79 \times 10^{-178}$	65,7%	26,3%	8,1%
Length & Completeness	6817	3788033,5	$< 10^{-300}$	65,4%	28,5%	6,0%
Guidelines	7639	3721414,0	$< 10^{-300}$	61,0%	37,7%	1,4%
Limitations	3365	1385458,5	$8,40 \times 10^{-148}$	84,2%	13,1%	2,7%
Examples	1261	151287,0	$1,44 \times 10^{-82}$	95,5%	3,9%	0,6%

Os seis atributos apresentam diferenças estatisticamente significativas entre os cenários ($p < 0{,}05$). Como o número de pares é elevado, até diferenças pequenas tendem a ser significativas, e a transição de quartil ajuda a dimensioná-las: *Purpose*, *Parameter Explanation*, *Length & Completeness* e *Guidelines* mudam de faixa em 31,8%, 34,4%, 34,5% e 39,1% das ferramentas, respectivamente, enquanto *Limitations* e *Examples* mudam em apenas 15,8% e 4,5%. Esses dois atributos são também os que têm menos pares com nota alterada (n), e, em todos os atributos, subir de faixa é mais frequente do que descer.

6. Ameaças à Validade

Validade de construto. Não há, neste estudo, um padrão-ouro humano de qualidade das descrições: a precisão da avaliação é inferida a partir do mecanismo pelo qual a nota muda (RQ1), não medida contra um julgamento de referência. Assim, demonstra-se que a mudança de nota tem causa identificável no código, mas não que a nota resultante seja, em sentido absoluto, mais correta. Além disso, tanto as avaliações quanto as etapas da análise qualitativa (síntese das justificativas, indução de categorias e classificação) são

conduzidas por LLMs, e a verificação manual compara a categoria atribuída com a justificativa sintetizada e o *reasoning* dos juízes, não diretamente com o código-fonte. O Gemini 3.5 Flash-Lite atua como juiz e também como sintetizador das justificativas, o que pode favorecer a sua própria leitura dos casos.

Validade interna. O código fornecido ao avaliador é obtido por uma heurística de resolução de chamadas baseada em nome, não em tipos, limitada a três níveis de profundidade e a 32.000 caracteres (Apêndice A). Parte da lógica relevante pode, portanto, ficar de fora, ou trechos de funções homônimas podem ser incluídos por engano, embora resoluções ambíguas sejam sinalizadas. As respostas dos juízes não são determinísticas, e a classificação das justificativas em categorias foi confirmada em 71,2% das ferramentas já verificadas, com discordâncias concentradas entre categorias de fronteira próxima. Por outro lado, a amostra de divergência confirmada exige que todos os juízes concordem que a nota mudou, o que a torna conservadora frente ao ruído de um juiz isolado.

Validade externa. A amostra restringe-se a 206 repositórios públicos do GitHub com ao menos 100 estrelas, concentrados em Python e TypeScript, e três repositórios respondem por 46,6% das ferramentas, o que pode enviesar resultados agregados. Os resultados também refletem três modelos específicos em uma versão e momento determinados; como os próprios juízes divergem na direção do efeito (RQ2), não é possível generalizar o comportamento observado para outros avaliadores.

Validade de conclusão. Com milhares de pares por atributo, até diferenças pequenas resultam em p-valores muito baixos; por isso, o Teste de Wilcoxon é acompanhado da transição de quartil, que dimensiona o efeito. Não foi aplicada correção para comparações múltiplas, mas, como todos os p-valores são inferiores a 10^{-80}, uma correção de Bonferroni para seis testes ($\alpha = 0{,}0083$) não alteraria nenhuma conclusão. Por fim, a média entre juízes, usada na RQ3, pode atenuar efeitos de sentidos opostos, como a redução de notas pelo DeepSeek frente ao aumento pelo Gemini e pelo Qwen3, razão pela qual a RQ2 apresenta os resultados por juiz.

7. Conclusão e Trabalhos Futuros

Este trabalho investigou se a avaliação da qualidade de descrições de ferramentas MCP é influenciada pela disponibilização do código da ferramenta ao avaliador. Para isso, 12.171 ferramentas de 206 servidores MCP foram extraídas, junto com o código alcançado por sua árvore de chamadas, e avaliadas por três LLMs segundo a rubrica de seis componentes de Hasan et al. (2026), com e sem acesso ao código.

Quanto à RQ1, na maior parte da amostra de divergência confirmada (89,2%) a mudança de nota decorre de um achado concreto no código, sobretudo parâmetros e opções não documentados, modos de falha não divulgados e detalhes da estrutura de saída; isso é consistente com um ganho de precisão, embora não o comprove sem um padrão-ouro humano. Quanto à RQ2, a direção do efeito depende do juiz: o DeepSeek tende a reduzir as notas, enquanto o Gemini e o Qwen3 tendem a elevá-las; *Purpose* é o atributo mais afetado e *Limitations* e *Examples* os menos afetados. Quanto à RQ3, as diferenças são estatisticamente significativas nos seis atributos, com mudança de faixa de quartil entre 31,8% e 39,1% das ferramentas em *Purpose*, *Parameter Explanation*, *Length & Completeness* e *Guidelines*.

Conclui-se que o código altera de forma consistente a avaliação das descrições e

que essa alteração, na maioria dos casos, tem origem em informações que a descrição omite. A principal contribuição do trabalho é mostrar que avaliar descrições MCP isoladamente deixa de captar uma parcela relevante de suas deficiências, e oferecer uma caracterização empírica, em 16 categorias, do tipo de informação que o código revela.

Como trabalhos futuros, propõe-se: (i) construir um padrão-ouro humano sobre uma amostra de ferramentas, permitindo medir diretamente se a avaliação com código é mais precisa; (ii) verificar se as deficiências reveladas pelo código afetam o desempenho real de agentes, por exemplo em *benchmarks* baseados em execução como o MCP-Universe [Luo et al. 2025]; (iii) usar o código para reescrever automaticamente as descrições e avaliar o ganho de qualidade resultante; (iv) medir a concordância entre avaliadores na análise qualitativa, com um segundo codificador independente e o coeficiente Kappa por categoria; e (v) aprimorar a extração do código, com resolução de chamadas baseada em tipos e profundidade adaptativa.

Referências

Borges, H., Hora, A., and Valente, M. T. (2016). Predicting the popularity of github repositories. In *Proceedings of the 12th International Conference on Predictive Models and Data Analytics in Software Engineering*, pages 1–10.

Braun, V. and Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2):77–101.

Chaokromthong, K., Sintao, N., et al. (2021). Sample size estimation using yamane and cochran and krejcie and morgan and green formulas and cohen statistical power analysis by g* power and comparisions. *Apheit international journal of interdisciplinary social sciences and technology*, 10(2):76–86.

Dong, Y., Li, Z., Tian, Y., Sun, C., Godfrey, M. W., and Nagappan, M. (2023). Bash in the wild: Language usage, code smells, and bugs. *ACM Trans. Softw. Eng. Methodol.*, 32(1).

Femmer, H., Méndez Fernández, D., Wagner, S., and Eder, S. (2017). Rapid quality assurance with requirements smells. *Journal of Systems and Software*, 123:190–213.

Guo, H., Hao, Y., Zhang, Y., Xu, M., Lv, P., Chen, J., and Cheng, X. (2025). A Measurement Study of Model Context Protocol Ecosystem. *arXiv e-prints*.

Hasan, M. M., Li, H., Fallahzadeh, E., Rajbahadur, G. K., Adams, B., and Hassan, A. E. (2026). Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers. *ACM Trans. Softw. Eng. Methodol.* Just Accepted.

Hsieh, H.-F. and Shannon, S. E. (2005). Three approaches to qualitative content analysis. *Qualitative Health Research*, 15(9):1277–1288.

ISO/IEC (2011). Iso/iec 25010:2011 – systems and software engineering – systems and software quality requirements and evaluation (square) – system and software quality models. Technical report, International Organization for Standardization, Geneva, Switzerland.

ISO/IEC/IEEE (2017). Iso/iec/ieee 24765:2017 – systems and software engineering – vocabulary. Technical report, International Organization for Standardization, Geneva, Switzerland.

Khan, J. Y., Khondaker, M. T. I., Uddin, G., and Iqbal, A. (2021). Automatic detection of five api documentation smells: Practitioners' perspectives. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 318–329.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

LLC, A. (2026). The model context protocol: A status report for enterprise customer experience leaders. White paper, Avaya.

Luo, Z., Shen, Z., Yang, W., Zhao, Z., Jwalapuram, P., Saha, A., Sahoo, D., Savarese, S., Xiong, C., and Li, J. (2025). MCP-Universe: Benchmarking Large Language Models with Real-World Model Context Protocol Servers. *arXiv e-prints*.

Mehedi Hasan, M., Li, H., Krishnan Rajbahadur, G., Adams, B., and Hassan, A. E. (2026). Model Context Protocol (MCP) Tool Descriptions Are Smelly! Towards Improving AI Agent Efficiency with Augmented MCP Tool Descriptions. *arXiv e-prints*.

National Security Agency (2026). Model context protocol (mcp): Security design considerations for ai-driven automation. Cybersecurity report, NSA Cybersecurity Information.

Pressman, R. S. and Maxim, B. R. (2016). *Engenharia de Software: Uma Abordagem Profissional*. AMGH, Porto Alegre, 8 edition.

Shi, Y., Zhang, X., Zhang, X., Shen, X., Ouyang, H., Qiu, H., Zhang, M., and Yang, M. (2026). Description-code inconsistency in real-world mcp servers: Measurement, detection, and security implications. *arXiv preprint arXiv:2606.04769*.

Taraghi, M., Morovati, M. M., and Khomh, F. (2026). Real faults in model context protocol (mcp) software: a comprehensive taxonomy. *arXiv preprint arXiv:2603.05637*.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J., and Stoica, I. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S., editors, *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc.

Apêndice

A. Algoritmo de Construção do Grafo de Chamadas e Composição do Contexto de Código

Este apêndice detalha o algoritmo desenvolvido para a construção do grafo de chamadas (*call graph*) de cada ferramenta MCP e para a composição do trecho de código usado como contexto adicional no cenário de avaliação que inclui código (Seção 4). O algoritmo é heurístico e baseado em análise estática via tree-sitter: a partir da função que implementa a própria ferramenta, resolve por nome as chamadas feitas dentro dela, até três níveis de profundidade, distinguindo chamadas resolvidas dentro do próprio repositório de chamadas externas ou dinamicamente determinadas, conforme ilustrado na Figura 4.

Procedimento de Extração

O algoritmo opera em três fases distintas, executadas uma vez por repositório:

1. **Indexação estática do repositório:** são construídos, a partir de todo o código do repositório, um índice de definições (nome simples e nome qualificado, `Classe.método`, de toda função/método encontrado, com referência ao seu corpo) e, por arquivo, um índice de *imports* (alias local usado no arquivo → módulo e nome original de onde veio). Ambos são heurísticas de escopo léxico, não uma resolução completa de símbolos.
2. **Construção recursiva do grafo de chamadas:** a árvore de cada ferramenta é construída a partir da função que a implementa (nível 1). Para cada chamada encontrada no corpo de uma função, uma heurística de resolução por nome é aplicada em ordem, valendo a primeira que casar: (i) receptor `self`/`this` → método da própria classe; (ii) mesmo nome definido no mesmo arquivo → função auxiliar local; (iii) receptor casa com um alias de *import* conhecido → busca no módulo importado; (iv) nome único em todo o repositório → resolve diretamente; (v) não encontrado em lugar nenhum (biblioteca externa, *stdlib*, ou chamada genuinamente dinâmica) → tratado como folha externa, sem recursão. Uma chamada resolvida vira um novo nó, expandido recursivamente até o nível 3; nós de nível 3 continuam sendo chamadas normalmente resolvidas, mas suas próprias chamadas internas não são mais exploradas. Esse limite é uma decisão de engenharia: contém o tamanho da árvore a um custo previsível e elimina, por construção, qualquer risco de laço infinito em recursão direta ou mútua, sem exigir detecção de ciclo separada. Quando mais de um candidato válido é encontrado num mesmo passo (nome duplicado em escopos distintos), o desempate é feito pela menor distância de diretório até o arquivo de origem da chamada, e o nó resultante permanece explicitamente sinalizado como *ambíguo* na estrutura de dados, em vez de ser tratado como uma resolução certa.
3. **Composição do contexto de código:** no cenário de avaliação que inclui código, a árvore é percorrida em pré-ordem e, para cada nó resolvido, o trecho de código correspondente (linhas de início e fim exatas) é lido diretamente do repositório clonado em disco e anotado com um comentário indicando o nível na árvore, a chamada de origem e a localização exata (arquivo:linhas). Nós externos/não resolvidos entram apenas como um comentário sinalizando a chamada não resolvida, sem trecho de código. Os trechos são concatenados na ordem de visita; o texto resultante é limitado a 32.000 caracteres antes de ser enviado ao avaliador, para conter o custo e a atenção do modelo: um corte por tamanho, não uma sumarização do conteúdo.

B. Prompts Utilizados na Análise Qualitativa das Divergências

Este apêndice reproduz, na íntegra e no idioma original (inglês, mesmo idioma em que a justificativa de cada juiz é produzida), os três *prompts* de sistema (*system instruction*) utilizados nas três etapas da análise qualitativa descrita na Seção 4.4. Em todos os casos, o modelo recebe o *prompt* de sistema reproduzido abaixo e, na mensagem do usuário, o objeto ou lista JSON de entrada descrito no próprio texto do *prompt*; a saída é restrita a um objeto ou lista JSON, sem texto adicional, por meio do mecanismo de saída estruturada do respectivo modelo: Gemini nas Etapas 1 e 3, e DeepSeek na Etapa 2.

Etapa 1: Geração de Justificativas

Aplicado individualmente a cada ferramenta da amostra de divergência confirmada (Seção 4.3), recebendo como entrada um objeto JSON com os campos `tool_uid`, `tool_name`, a lista `avaliacoes` (uma entrada por juiz e cenário, já restrita aos componentes da rubrica cuja nota divergiu) e `source_code`.

```
You are a research assistant helping analyze results from an undergraduate thesis ("Model Context Protocol (MCP): Avaliação da Qualidade de Descrições de Tools com Base em Contexto de Código-Fonte") that studies whether giving an LLM-as-judge access to a tool's source code changes its evaluation of the tool's natural-language description. Several judge models independently scored the SAME description on a 1-5 Likert scale across 6 rubric components (Purpose, Guidelines, Limitations, Parameter Explanation, Length & Completeness, Examples), once without the source code ("description_only") and once with it ("with_source"). The judges were instructed to first score the description without considering the source code, and then to use the source code to identify inconsistencies between the description and the implementation, adjusting the initial score accordingly. The instruction does not restrict the direction of that adjustment, so a score change may or may not be grounded in a concrete finding from the code.

You will receive one JSON object describing a single tool for which at least one component's score changed between the two scenarios, for every judge that evaluated it unanimously. It has:
- "tool_uid": a unique identifier for the tool.
- "tool_name": the tool's name.
- "avaliacoes": a list of per-judge, per-scenario entries. Each has "scenario" ("description_only" or "with_source"), "provider" (which judge model), and "scores" -- only the components whose score differs between the two scenarios for that judge, each with the numeric "score" and that judge's own "reasoning" explaining it.
- "source_code": the actual source snippet(s) the "with_source" judges saw.

Task: read every judge's reasoning for its changed component(s), cross-reference it against "source_code", and write ONE justification (3-5 sentences) explaining WHAT the source code revealed that the plain-text description did not -- e.g. an undocumented parameter, an undisclosed behavior/output, a missing limitation or error case, or a direct contradiction between the description and the implementation. If judges disagree on the reason, or if a judge's reasoning gives no concrete evidence from the code (just a vaguer/stricter
```

```
    reading of the same text), say so plainly instead of
    inventing a justification.
10
11  Ground every claim in specific details from the reasoning and/or
        the source code. Do not just restate "the score changed" or
    copy one judge's reasoning verbatim -- synthesize across
    judges. If different judges changed different components for
    different reasons, address each briefly, in one paragraph.
    Always write the justification in English, regardless of the
    reasoning's original language (it is already in English in
    this dataset, but do not drop this rule if that ever changes)
    .
12
13  Be conservative per component: a strong, well-evidenced finding
    for one judge/component (e.g. a parameter revealed to be
    hardcoded, or a disclosed error case) must NOT be stretched
    to also explain a different judge's or a different component'
    s change unless that other reasoning independently supports
    it. When a specific score change's reasoning is vague or
    generic (e.g. "now seems clearer," "provides more context,"
    with no concrete detail cited), say explicitly that no clear
    code-based reason is evident for that particular change
    instead of inferring one. Never introduce terminology, named
    standards, formulas, or mechanisms that do not literally
    appear in the reasoning or in "source_code" -- even when they
        would be plausible, typical domain knowledge for a tool like
        this one. If you are not sure a claim is directly supported
    by the given text, leave it out.
14
15  Output ONLY one JSON object, no text before or after it, in
    exactly this shape:
16  {"tool_uid": "<the tool_uid from the input, verbatim>", "
    justification": "<your justification>"}
```

Listing 1. Prompt de sistema – geração de justificativas

Etapa 2: Identificação Indutiva de Categorias

Aplicado uma única vez, recebendo como entrada o conjunto completo de justificativas produzidas na Etapa 1, no formato de uma lista JSON de objetos {"tool_uid": ..., "justification": ...}.

```
1   You are a qualitative research assistant conducting a thematic
    analysis for an undergraduate thesis ("Model Context Protocol
        (MCP): Avaliação da Qualidade de Descrições de Tools com
    Base em Contexto de Código-Fonte") that studies whether
    giving an LLM-as-judge access to a tool's source code changes
        its evaluation of the tool's natural-language description.
    For every tool where every judge that evaluated it changed at
        least one rubric score between the "description_only" and "
    with_source" scenarios, a short English justification was
```

```
    already written, explaining WHAT the source code revealed
    that the plain-text description did not.
2
3 You will receive a JSON array of {"tool_uid": ..., "
    justification": ...} objects -- one per tool, every
    justification independently written from that tool's own
    judge reasoning and source code.
4
5 Task: perform an inductive thematic analysis (open coding,
    following Braun & Clarke's six-phase method: familiarization,
     initial coding, searching for themes, reviewing themes,
    defining and naming themes) across ALL justifications to
    identify the recurring categories of WHAT KIND of thing the
    source code tends to reveal that the description does not. Do
     not start from a predefined list -- let the categories
    emerge bottom-up from patterns across the justifications
    themselves. Each category must describe a MECHANISM (what the
     code exposed), never a rubric component (Purpose, Guidelines
    , Limitations, Parameter Explanation, Length & Completeness,
    Examples) and never a direction (score went up/down) -- e.g.
    "the code reveals a hardcoded parameter the description
    presents as configurable" is a mechanism; "Limitations score
    decreased" is not.
6
7 Requirements for the final set of categories:
8 - Between 5 and 15 categories. Fewer undersegments genuinely
    distinct patterns; more usually means two categories should
    be merged, or a category is really a one-off outlier rather
    than a recurring pattern.
9 - Each category must be grounded in multiple justifications
    across different tools -- never create a category for a
    single idiosyncratic case.
10 - Categories must be mutually distinguishable: if two candidate
    categories would classify the same justification the same way
     almost every time, merge them into one.
11 - Name each category concisely (2-5 words, a short analytic
    label, Title Case).
12 - Each description must state, in 2-3 sentences: (a) what the
    category captures, in terms of the specific kind of
    discrepancy or disclosure found in the code, and (b) one
    illustrative, paraphrased example pattern drawn from the
    justifications -- never quote or name a specific tool_uid.
13
14 Output ONLY a JSON array, no text before or after it, in exactly
     this shape:
15 [{"category": "<Title Case name>", "description": "<2-3 sentence
     description>"}, ...]
16
```

```
Write everything in English.
```

Listing 2. Prompt de sistema – identificação indutiva de categorias

Etapa 3: Classificação Dedutiva por Categoria

Aplicado individualmente a cada ferramenta da amostra de divergência confirmada. O trecho reproduzido como `<lista de categorias da Etapa 2, uma por linha>` é substituído, em tempo de execução, pela lista de categorias produzida na Etapa 2 (nome e descrição de cada uma); `"none"` é a categoria de reserva explícita, utilizada quando nenhuma das categorias da lista fixa corresponde à justificativa analisada.

```
You are a qualitative research assistant performing directed
   content analysis for an undergraduate thesis ("Model Context
   Protocol (MCP): Avaliação da Qualidade de Descrições de Tools
    com Base em Contexto de Código-Fonte") that studies whether
   giving an LLM-as-judge access to a tool's source code changes
    its evaluation of the tool's natural-language description.

A prior inductive thematic analysis identified the following
   FIXED set of categories, describing the MECHANISM by which
   the source code revealed something the description did not.
   Treat this list as closed -- do not invent new categories,
   rename any, or merge/split any, even if a justification seems
    to need a slightly different label:

<lista de categorias da Etapa 2, uma por linha>

You will receive one JSON object: {"tool_uid": ..., "
   justification": ...}. Assign every category above whose
   description genuinely matches what THIS justification says --
    it may match more than one category (do not force a single
   label), and should match zero categories only if none of them
    genuinely fits (in that case, output the literal string "
   none" as the sole entry, never an empty list).

Base your assignment only on this justification's own content,
   not on any other tool. Output ONLY one JSON object, no text
   before or after it, in exactly this shape:
{"tool_uid": "<copied verbatim from the input>", "categories":
   ["<matching category name(s), or \"none\">"]}
```

Listing 3. Prompt de sistema – classificação dedutiva por categoria

C. Categorias de Mecanismo

A Tabela 4 apresenta as 16 categorias de mecanismo produzidas pela análise temática indutiva (Seção 4.4), cujas frequências estão na Tabela 2.

Tabela 4. Categorias de mecanismo identificadas pela análise temática indutiva sobre a amostra de divergência confirmada

Categoria	Descrição
Superfície de Entrada Não Documentada	A assinatura ou o esquema de entrada do código aceita parâmetros, filtros, identificadores de sessão ou combinações obrigatórias que a descrição nunca menciona.
Modos de Falha e Erros Não Divulgados	Tratamentos de erro, exceções, falhas de dependência e mensagens de estado vazio da implementação não são antecipados pela descrição.
Estrutura Concreta da Saída	A estrutura exata do retorno (campos, aninhamento, formatação, variações por sucesso/falha) é mais específica do que a descrição sugere.
Regras de Validação e Contratos de Formato Não Divulgados	Conjuntos de valores permitidos, formatos, regras de exclusão mútua ou campos obrigatórios fixados no código não são declarados na descrição.
Limites Operacionais, Padrões e Paginação Não Divulgados	Tetos, truncamentos, *timeouts*, paginação e valores padrão não declarados são impostos pela implementação sem menção na descrição.
Contexto de Fluxo e Ativação	O código revela pré-requisitos de estado de sessão, ordem de chamadas entre ferramentas irmãs, ou contexto ambiente que a descrição não articula.
Escopo de Processamento e Mecânica Interna Subestimados	A implementação realiza bem mais processamento ou mecânica interna (múltiplas sub-chamadas, *cache*, chamadas de rede, lógica assíncrona) do que a descrição sugere.
Confirmação de Encapsulamento Trivial	O código é um encaminhamento mínimo cujo comportamento já correspondia exatamente ao texto, sem revelar nada novo.
Valores Fixos e Padrões Divergentes	Um valor apresentado como configurável está, na realidade, fixado no código, ou o valor padrão documentado diverge do valor efetivamente usado.
Dependências e Pré-requisitos de Ambiente Ocultos	Serviços externos, variáveis de ambiente, credenciais, binários ou configurações exigidos pelo código não são mencionados na descrição.
Semântica de Correspondência e Normalização de Entrada	O código processa entradas de forma não óbvia (correspondência por substring, normalização de apelidos, resolução difusa de nomes) sem que a descrição mencione essa semântica.
Controles de Acesso, Papel e Autorização	Permissões, autenticação, verificações de papel/administrador e outras restrições de acesso aplicadas no código estão ausentes da descrição.
Parâmetros Fantasmas ou Divergentes	A descrição documenta parâmetros que o código não possui, ou que foram renomeados, removidos ou substituídos na implementação.
Funcionalidade Anunciada Não Implementada	O código expõe um *stub*, um retorno fixo, ou simplesmente não implementa a funcionalidade que a descrição promete.
Sistema-Alvo Divergente	O código revela uma plataforma, formato de arquivo ou granularidade de recurso diferente da que a descrição declara.
Efeitos Colaterais Não Divulgados	O código produz efeitos colaterais (exclusões em cascata, escrita de arquivos, notificações) além da ação anunciada pela descrição.